

FreeRTOS内存管理 基于STM32

目录

[一、内存管理的基本概念](#)

[二、内存管理的应用场景](#)

[三、heap_4.c](#)

[1.内存申请函数 pvPortMalloc\(\)](#)

[2.内存释放函数 vPortFree\(\)](#)

[四、内存管理的实验](#)

[五、内存管理的实验现象](#)

在计算系统中，变量、中间数据一般存放在系统存储空间中，只有在实际使用时才将 它们从存储空间调入到中央处理器内部进行运算。通常存储空间可以分为两种：内部存储空间和外部存储空间。内部存储空间访问速度比较快，能够按照变量地址随机地访问，也 就是我们通常所说的 RAM（随机存储器），或电脑的内存；而外部存储空间内所保存的内容相对来说比较固定，即使掉电后数据也不会丢失，可以把它理解为电脑的硬盘。在这一章中我们主要讨论内部存储空间（RAM）的管理——内存管理。

[FreeRTOS](#) 操作系统将内核与内存管理分开实现，操作系统内核仅规定了必要的内存管理函数原型，而不关心这些内存管理函数是如何实现的，所以在 FreeRTOS 中提供了多种内存分配算法（分配策略），但是上层接口（API）却是统一的。这样做可以增加系统的灵活性：用户可以选择对自己更有利的内存管理策略，在不同的应用场合使用不同的内存分配策略。

在嵌入式程序设计中[内存分配](#)应该是根据所设计系统的特点来决定选择使用动态内存分配还是静态内存分配算法，一些可靠性要求非常高的系统应选择使用静态的，而普通的业务系统可以使用动态来提高内存使用效率。静态可以保证设备的可靠性但是需要考虑内存上限，内存使用效率低，而动态则是相反。

FreeRTOS 内存管理模块管理用于系统中内存资源，它是操作系统的核心模块之一。主要包括内存的初始化、分配以及释放。

很多人会有疑问，为什么不直接使用 C 标准库中的内存管理函数呢？在电脑中我们可以用 malloc()和 free()这两个函数动态的分配内存和释放内存。但是，在嵌入式实时操作系统中，调用 malloc()和 free()却是危险的，原因有以下几点：

- 1.这些函数在小型嵌入式系统中并不总是可用的，小型嵌入式设备中的 RAM 不足。
- 2.它们的实现可能非常的大，占据了相当大的一块代码空间。
- 3.他们几乎都不是安全的。
- 4.它们并不是确定的，每次调用这些函数执行的时间可能都不一样。
- 5.它们有可能产生碎片。
- 6.这两个函数会使得链接器配置得复杂。
- 7.如果允许堆空间的生长方向覆盖其他变量占据的内存，它们会成为 debug 的灾难。

在一般的实时嵌入式系统中，由于实时性的要求，很少使用虚拟内存机制。所有的内存都需要用户参与分配，直接操作物理内存，所分配的内存不能超过系统的物理内存，所有的系统堆栈的管理，都由用户自己管理。

同时，在嵌入式实时操作系统中，对内存的分配时间要求更为苛刻，分配内存的时间必须是确定的。一般内存管理算法是根据需要存储的数据的长度在内存中寻找一个与这段数据相适应的空闲内存块，然后将数据存储在里面。而寻找这样一个空闲内存块所耗费的时间是不确定的，因此对于实时系统来说，这就是不可接受的，实时系统必须要保证内存块的分配过程在可预测的确定时间内完成，否则实时任务对外部事件的响应也将变得不可确定。

而在嵌入式系统中，内存是十分有限而且是十分珍贵的，用一块内存就少了一块内存，而在分配中随着内存不断被分配和释放，整个系统内存区域会产生越来越多的碎片，因为在使用过程中，申请了一些内存，其中一些释放了，导致内存空间中存在一些小的内存块，它们地址不连续，不能够作为一整块的大内存分配出去，所以一定会在某个时间，系统已经无法分配到合适的内存了，导致系统瘫痪。其实系统中实际是还有内存的，但是因为小块的内存的地址不连续，导致无法分配成功，所以我们需要一个优良的内存分配算法来避免这种情况的出现。

不同的嵌入式系统具有不同的内存配置和时间要求。所以单一的内存分配算法只可能 适合部分应用程序。因此，FreeRTOS 将内存分配作为可移植层面（相对于基本的内核代码 部分而言），FreeRTOS 有针对性的提供了不同的内存分配管理算法，这使得应用于不同场 景的设备可以选择适合自身内存算法。

FreeRTOS 对内存管理做了很多事情，FreeRTOS 的 V9.0.0 版本为我们提供了 5 种内存 管理算法，分别是 heap_1.c、heap_2.c、heap_3.c、heap_4.c、heap_5.c，源文件存放于 FreeRTOS\Source\portable\MemMang 路径下，在使用的时候选择其中一个添加到我们的工 程中去即可。

FreeRTOS 的内存管理模块通过对内存的申请、释放操作，来管理用户和系统对内存的 使用，使内存的利用率和使用效率达到最优，同 时最大限度地解决系统可能产生的内存碎片问题。

二、内存管理的应用场景

首先，在使用内存分配前，必须明白自己在做什么，这样做与其他的方法有什么不同， 特别是会产生哪些负面影响，在自己的产品面 前，应当选择哪种分配策略。

内存管理的主要工作是动态划分并管理用户分配好的内存区间，主要是在用户需要使 用大小不等的内存块的场景中使用，当用户需要分 配内存时，可以通过操作系统的内存申 请函数索取指定大小内存块，一旦使用完毕，通过动态内存释放函数归还所占用内存，使 之可以重 复使用（heap_1.c 的内存管理除外）。

例如我们需要定义一个 float 型数组：floatArr[];

但是，在使用数组的时候，总有一个问题困扰着我们：数组应该有多大？在很多的情 况下，你并不能确定要使用多大的数组，可能为了 避免发生错误你就需要把数组定义得足 够大。即使你知道想利用的空间大小，但是如果因为某种特殊原因空间利用的大小有增加 或者减 少，你又必须重新去修改程序，扩大数组的存储范围。这种分配固定大小的内存分 配方法称之为静态内存分配。这种内存分配的方法存在 比较严重的缺陷，在大多数情况下会浪费大量的内存空间，在少数情况下，当你定义的数组不够大时，可能引起下标越界错 误，甚至导致严重后果。

我们用动态内存分配就可以解决上面的问题。所谓动态内存分配就是指在程序执行的 过程中动态地分配或者回收存储空间的分配内存的 方法。动态内存分配不象数组等静态内 存分配方法那样需要预先分配存储空间，而是由系统根据程序的需要即时分配，且分配的 大小就是 程序要求的大小。

三、heap_4.c

我们这里只讲heap_4.c因为这个最常用。内存分配时需要的总的堆空间由文件FreeRTOSConfig.h中的宏 configTOTAL_HEAP_SIZE 配置，单位为字。通过调用函数 xPortGetFreeHeapSize() 我们可 以知道还剩下多少内存没有使用，但是并不包 括内存碎片。这样一来我们可以实时的调整 和优化 configTOTAL_HEAP_SIZE 的大小。

heap_4.c 方案的空闲内存块也是以单链表的形式连接起来的，BlockLink_t 类型的局部 静态变量 xStart 表示链表头，但 heap_4.c 内存 管理方案的链表尾部则保存在内存堆空间最 后位置，并使用 BlockLink_t 指针类型局部静态变量 pxEnd 指向这个区域（而 heap_2.c 内 存管理方案则使用 BlockLink_t 类型的静态变量 xEnd 表示链表尾）

heap_4.c 内存管理方案的空闲块链表不是以内存块大小进行排序的，而是以内存块起 始地址大小排序，内存地址小的在前，地址大的 在后，因为 heap_4.c 方案还有一个内存合 并算法，在释放内存的时候，假如相邻的两个空闲内存块在地址上是连续的，那么就可以 合并 为一个内存块，这也是为了适应合并算法而作的改变。

heap_4.c 方案具有以下特点：

- 1、可用于重复删除任务、队列、信号量、互斥量等的应用程序
- 2、可用于分配和释放随机字节内存的应用程序，但并不像 heap2.c 那样产生严重的内 存碎片。
- 3、具有不确定性，但是效率比标准 C 库中的 malloc 函数高得多。

1.内存申请函数 pvPortMalloc()

heap_4.c 方案的内存申请函数与 heap_2.c 方案的内存申请函数大同小异，同样是从链 表头 xStart 开始遍历查找合适的内存块，如果某 个空闲内存块的大小能容得下用户要申请 的内存，则将这块内存取出用户需要内存空间大小的部分返回给用户，剩下的内存块组成 一个新的 空闲块，按照空闲内存块起始地址大小顺序插入到空闲块链表中，内存地址小的 在前，内存地址大的在后。在插入到空闲内存块链表的过程中，系统还会执行合并算法将 地址相邻的内存块进行合并：判断这个空闲内存块是相邻的空闲内存块合并成一个大内 存块，如果可以 则合并，合并算法是 heap_4.c 内存管理方案和 heap_2.c 内存管理方案最大的 不同之处，这样一来，会导致的内存碎片就会大大减少，内 存管理方案适用性就很强，能 一样随机申请和释放内存的应用中，灵活性得到大大的提高，heap_4.c 内存初始化完成示意图具体见图1。

heap_4.c 的内存 申请源码

```

1. void *pvPortMalloc( size_t xWantedSize )
2. {
3.     BlockLink_t *pxBlock, *pxPreviousBlock, *pxNewBlockLink;
4.     void *pvReturn = NULL;
5.
6.     vTaskSuspendAll();
7.     {
8.         /* If this is the first call to malloc then the heap will require
9.         initialisation to setup the list of free blocks. */
10.        if( pxEnd == NULL )
11.        {
12.            prvHeapInit();
13.        }
14.        else
15.        {
16.            mtCOVERAGE_TEST_MARKER();
17.        }
18.
19.        /* Check the requested block size is not so large that the top bit is
20.        set. The top bit of the block size member of the BlockLink_t structure
21.        is used to determine who owns the block - the application or the
22.        kernel, so it must be free. */
23.        if( ( xWantedSize & xBlockAllocatedBit ) == 0 )
24.        {
25.            /* The wanted size is increased so it can contain a BlockLink_t
26.            structure in addition to the requested amount of bytes. */
27.            if( xWantedSize > 0 )
28.            {
29.                xWantedSize += xHeapStructSize;
30.
31.                /* Ensure that blocks are always aligned to the required number
32.                of bytes. */
33.                if( ( xWantedSize & portBYTE_ALIGNMENT_MASK ) != 0x00 )
34.                {
35.                    /* Byte alignment required. */
36.                    xWantedSize += ( portBYTE_ALIGNMENT - ( xWantedSize & portBYTE_ALIGNMENT_MASK ) );
37.                    configASSERT( ( xWantedSize & portBYTE_ALIGNMENT_MASK ) == 0 );
38.                }
39.                else
40.                {
41.                    mtCOVERAGE_TEST_MARKER();
42.                }
43.            }
44.            else
45.            {
46.                mtCOVERAGE_TEST_MARKER();
47.            }
48.
49.            if( ( xWantedSize > 0 ) && ( xWantedSize <= xFreeBytesRemaining ) )
50.            {
51.                /* Traverse the list from the start (lowest address) block until
52.                one of adequate size is found. */
53.                pxPreviousBlock = &xStart;
54.                pxBlock = xStart.pxNextFreeBlock;
55.                while( ( pxBlock->xBlockSize < xWantedSize ) && ( pxBlock->pxNextFreeBlock != NULL ) )
56.                {
57.                    pxPreviousBlock = pxBlock;
58.                    pxBlock = pxBlock->pxNextFreeBlock;
59.                }
60.
61.                /* If the end marker was reached then a block of adequate size
62.                was not found. */
63.                if( pxBlock != pxEnd )
64.                {
65.                    /* Return the memory space pointed to - jumping over the
66.                    BlockLink_t structure at its start. */
67.                    pvReturn = ( void * ) ( ( ( uint8_t * ) pxPreviousBlock->pxNextFreeBlock ) + xHeapStructSize );
68.
69.                    /* This block is being returned for use so must be taken out
70.                    of the list of free blocks. */
71.                    pxPreviousBlock->pxNextFreeBlock = pxBlock->pxNextFreeBlock;
72.
73.                    /* If the block is larger than required it can be split into
74.                    two. */
75.                    if( ( pxBlock->xBlockSize - xWantedSize ) > heapMINIMUM_BLOCK_SIZE )
76.                    {
77.                        /* This block is to be split into two. Create a new
78.                        block following the number of bytes requested. The void
79.                        cast is used to prevent byte alignment warnings from the
80.                        compiler. */
81.                        pxNewBlockLink = ( void * ) ( ( ( uint8_t * ) pxBlock ) + xWantedSize );
82.                        configASSERT( ( ( ( size_t ) pxNewBlockLink ) & portBYTE_ALIGNMENT_MASK ) == 0 );
83.
84.                        /* Calculate the sizes of two blocks split from the
85.                        single block. */
86.                        pxNewBlockLink->xBlockSize = pxBlock->xBlockSize - xWantedSize;
87.                        pxBlock->xBlockSize = xWantedSize;
88.
89.                        /* Insert the new block into the list of free blocks. */
90.                        prvInsertBlockIntoFreeList( pxNewBlockLink );
91.                    }
92.                }
93.            }
94.            else
95.            {
96.                mtCOVERAGE_TEST_MARKER();
97.            }
98.        }
99.    }
100.    vTaskResumeAll();
101.    return pvReturn;
102.}

```

```

97.         xFreeBytesRemaining -= pxBlock->xBlockSize;
98.
99.         if( xFreeBytesRemaining < xMinimumEverFreeBytesRemaining )
100.         {
101.             xMinimumEverFreeBytesRemaining = xFreeBytesRemaining;
102.         }
103.         else
104.         {
105.             mtCOVERAGE_TEST_MARKER();
106.         }
107.
108.         /* The block is being returned - it is allocated and owned
109.         by the application and has no "next" block. */
110.         pxBlock->xBlockSize |= xBlockAllocatedBit;
111.         pxBlock->pxNextFreeBlock = NULL;
112.     }
113.     else
114.     {
115.         mtCOVERAGE_TEST_MARKER();
116.     }
117. }
118. else
119. {
120.     mtCOVERAGE_TEST_MARKER();
121. }
122. }
123. else
124. {
125.     mtCOVERAGE_TEST_MARKER();
126. }
127.
128.     traceMALLOC( pvReturn, xWantedSize );
129. }
130. ( void ) xTaskResumeAll();
131.
132. #if( configUSE_MALLOC_FAILED_HOOK == 1 )
133. {
134.     if( pvReturn == NULL )
135.     {
136.         extern void vApplicationMallocFailedHook( void );
137.         vApplicationMallocFailedHook();
138.     }
139.     else
140.     {
141.         mtCOVERAGE_TEST_MARKER();
142.     }
143. }
144. #endif
145.
146. configASSERT( ( ( ( size_t ) pvReturn ) & ( size_t ) portBYTE_ALIGNMENT_MASK ) == 0 );
147. return pvReturn;
148. }

```



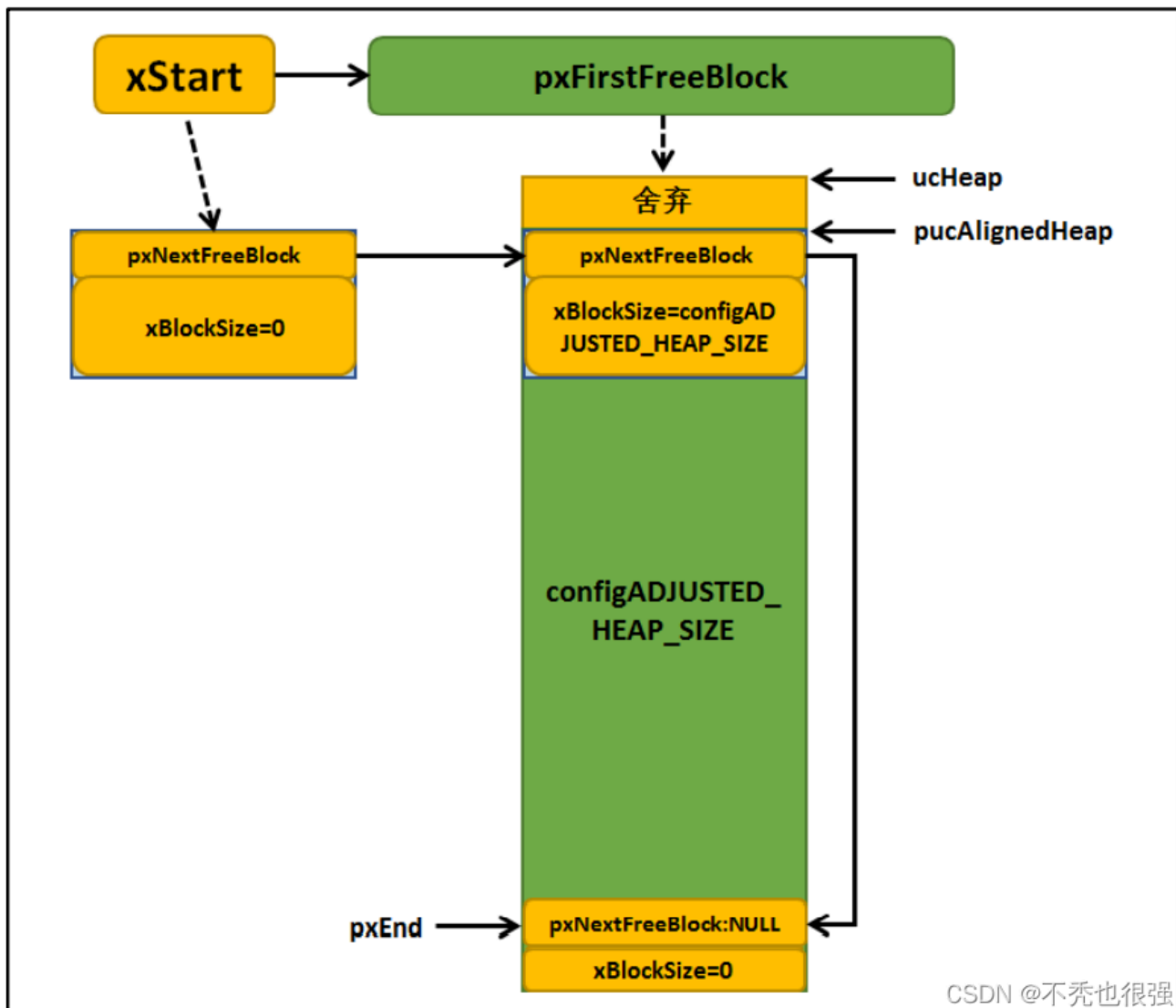


图1 内存初始化完成示意图

2.内存释放函数 vPortFree()

heap_4.c 内存管理方案的内存释放函数 vPortFree() 也比较简单，根据传入要释放的内存块地址，偏移之后找到链表节点，然后将这个内存块插入到空闲内存块链表中，在内存块插入过程中会执行合并算法，这个我们已经在内存申请中讲过了（而且合并算法多用于释放内存中）。最后是将这个内存块标志为“空闲”（内存块节点的 xBlockSize 成员变量 最高位清 0）、再更新未分配的内存堆大小即可，下面来看看 vPortFree() 的源码实现过程。

```

1. void vPortFree( void *pv )
2. {
3.     uint8_t *puc = ( uint8_t * ) pv;
4.     BlockLink_t *pxLink;
5.
6.     if( pv != NULL )
7.     {
8.         /* The memory being freed will have an BlockLink_t structure immediately
9.         before it. */
10.        puc -= xHeapStructSize;
11.
12.        /* This casting is to keep the compiler from issuing warnings. */
13.        pxLink = ( void * ) puc;
14.
15.        /* Check the block is actually allocated. */
16.        configASSERT( ( pxLink->xBlockSize & xBlockAllocatedBit ) != 0 );
17.        configASSERT( pxLink->pxNextFreeBlock == NULL );
18.
19.        if( ( pxLink->xBlockSize & xBlockAllocatedBit ) != 0 )
20.        {
21.            if( pxLink->pxNextFreeBlock == NULL )
22.            {
23.                /* The block is being returned to the heap - it is no longer
24.                allocated. */
25.                pxLink->xBlockSize &= ~xBlockAllocatedBit;
26.
27.                vTaskSuspendAll();
28.                {
29.                    /* Add this block to the list of free blocks. */
30.                    xFreeBytesRemaining += pxLink->xBlockSize;

```

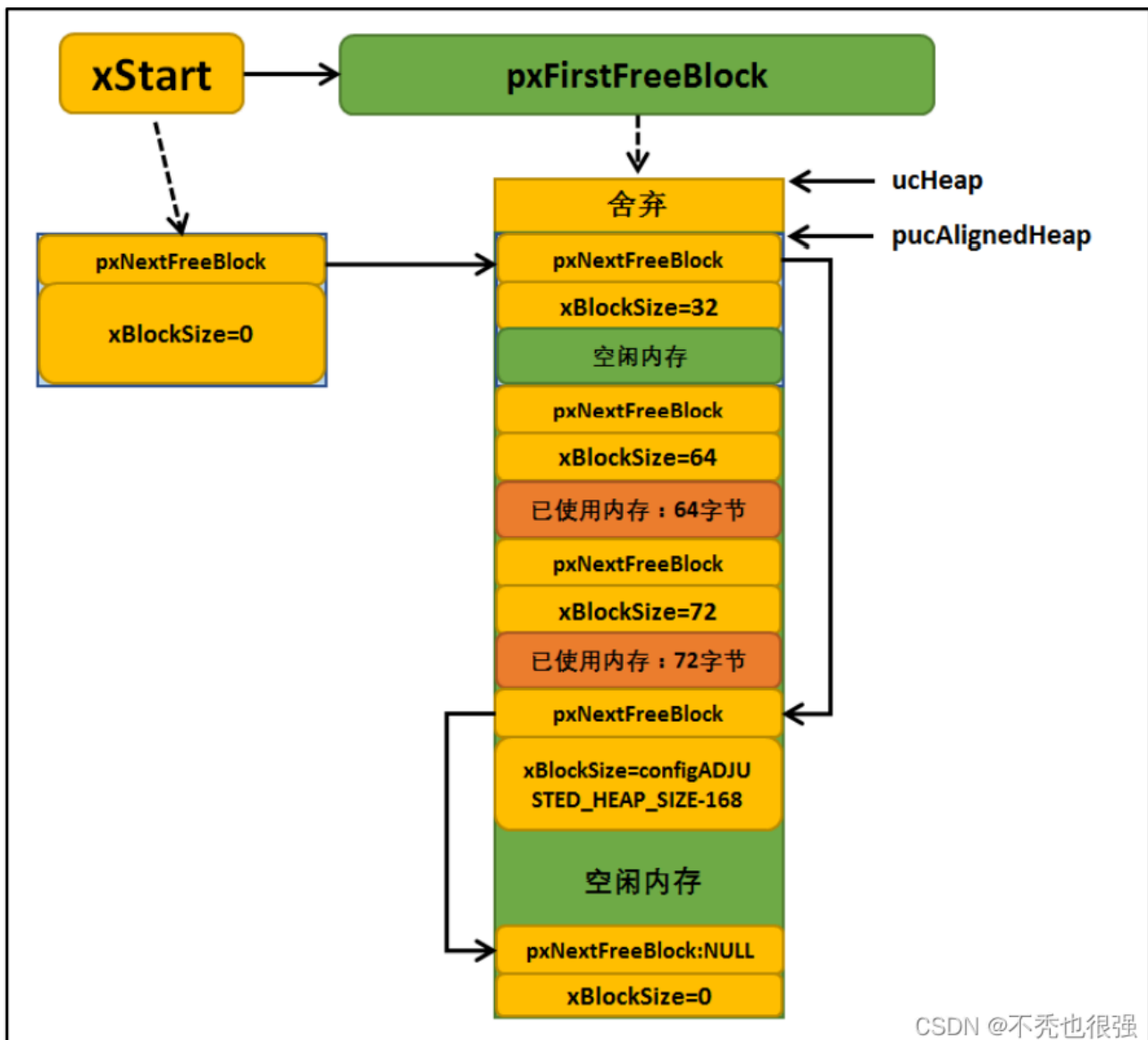
```

31.         traceFREE( pv, pxLink->xBlockSize );
32.         prvInsertBlockIntoFreeList( ( ( BlockLink_t * ) pxLink ) );
33.     }
34.     ( void ) xTaskResumeAll();
35. }
36. else
37. {
38.     mtCOVERAGE_TEST_MARKER();
39. }
40. }
41. else
42. {
43.     mtCOVERAGE_TEST_MARKER();
44. }
45. }
46. }

```



调用 `prvInsertBlockIntoFreeList()` 函数将释放的内存块添加到空闲内存块链表中，在这过程中，如果内存块可以合并就会进行内存块合并，否则就单纯插入空闲内存块链表（按内存地址排序）。按照内存释放的过程，当我们释放一个内存时，如果与它相邻的内存块都不是空闲的，那么该内存块并不会合并，只会被添加到空闲内存块链表中，其过程示意图具体见图 2。而如果某个时间段释放了另一个内存块，发现该内存块前面有一个空闲内存块与它在地址上是连续的，那么这两个内存块会合并成一个大的内存块，并插入空闲内存块链表中，其过程示意图具体见图 3，



图二释放一个内存块（无法合并）

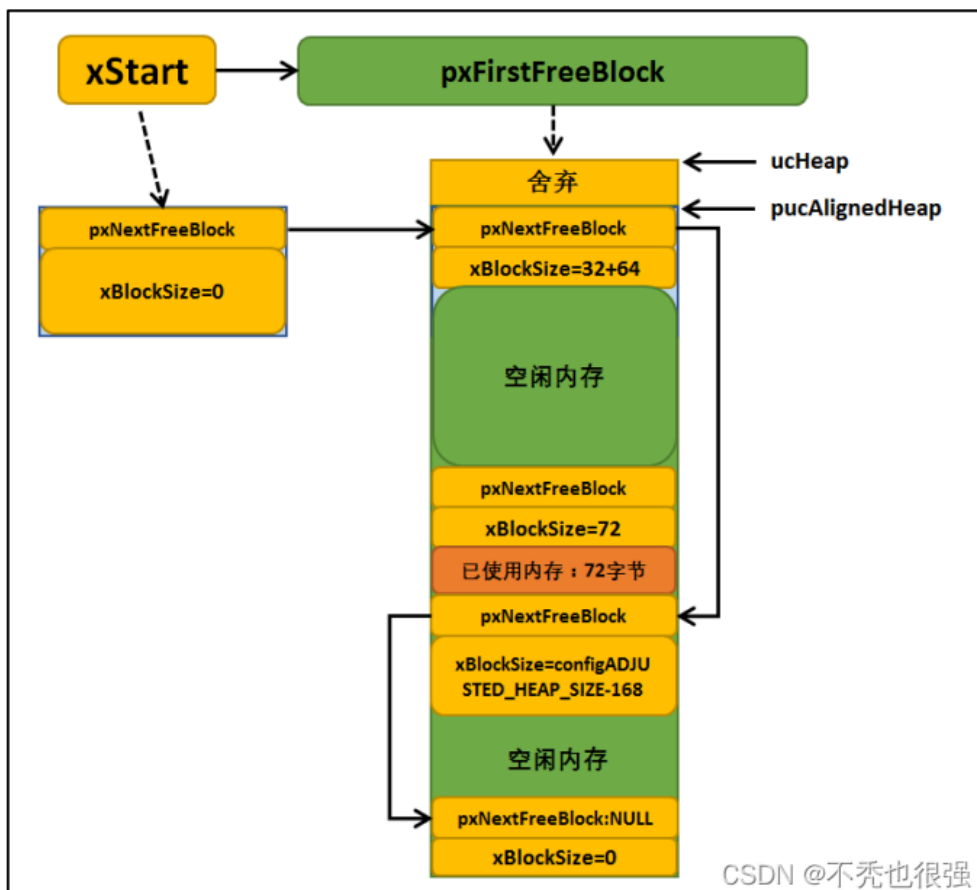


图3 释放一个内存块（可以合并）

四、内存管理的实验

内存管理实验使用 heap_4.c 方案进行内存管理测试，创建了两个任务，分别是 LED 任务与内存管理测试任务，内存管理测试任务通过检测按键是否按下来申请内存或释放内存，当申请内存成功就像该内存写入一些数据，如当前系统的时间等信息，并且通过串口输出相关信息；LED 任务是将 LED 翻转，表示系统处于运行状态。在不需要再使用内存时，注意要及时释放该段内存，避免内存泄露。

```

1. /* FreeRTOS头文件 */
2. #include "FreeRTOS.h"
3. #include "task.h"
4. /* 开发板硬件bsp头文件 */
5. #include "bsp_led.h"
6. #include "bsp_usart.h"
7. #include "bsp_key.h"
8. /***** 任务句柄 *****/
9. /*
10. * 任务句柄是一个指针，用于指向一个任务，当任务创建好之后，它就具有了一个任务句柄
11. * 以后我们要想操作这个任务都需要通过这个任务句柄，如果是自身的任务操作自己，那么
12. * 这个句柄可以为NULL。
13. */
14. static TaskHandle_t AppTaskCreate_Handle = NULL; /* 创建任务句柄 */
15. static TaskHandle_t LED_Task_Handle = NULL; /* LED_Task任务句柄 */
16. static TaskHandle_t Test_Task_Handle = NULL; /* Test_Task任务句柄 */
17.
18.
19.
20. /***** 全局变量声明 *****/
21. /*
22. * 当我们在写应用程序的时候，可能需要用到一些全局变量。
23. */
24. uint8_t *Test_Ptr = NULL;
25.
26.
27. /*
28. *****/
29. * 函数声明
30. *****/
31. /*
32. static void AppTaskCreate(void); /* 用于创建任务 */
33.
34. static void LED_Task(void* pvParameters); /* LED_Task任务实现 */
35. static void Test_Task(void* pvParameters); /* Test_Task任务实现 */
36.
37. static void BSP_Init(void); /* 用于初始化板载相关资源 */
38.
39. /*****
40. * @brief 主函数
41. * @param 无

```

```

42.  * @retval 无
43.  * @note   第一步：开发板硬件初始化
44.           第二步：创建APP应用任务
45.           第三步：启动FreeRTOS，开始多任务调度
46.  *****/
47. int main(void)
48. {
49.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
50.
51.     /* 开发板硬件初始化 */
52.     BSP_Init();
53.     printf("这是一个FreeRTOS内存管理实验\n");
54.     printf("按下KEY1申请内存，按下KEY2释放内存\n");
55.     /* 创建AppTaskCreate任务 */
56.     xReturn = xTaskCreate((TaskFunction_t)AppTaskCreate, /* 任务入口函数 */
57.                          (const char*) "AppTaskCreate", /* 任务名字 */
58.                          (uint16_t) 512, /* 任务栈大小 */
59.                          (void*) NULL, /* 任务入口函数参数 */
60.                          (UBaseType_t) 1, /* 任务的优先级 */
61.                          (TaskHandle_t*) &AppTaskCreate_Handle); /* 任务控制块指针 */
62.
63.     /* 启动任务调度 */
64.     if (pdPASS == xReturn)
65.         vTaskStartScheduler(); /* 启动任务，开启调度 */
66.     else
67.         return -1;
68.
69.     while(1); /* 正常不会执行到这里 */
70. }
71.
72. *****/
73. * @ 函数名 : AppTaskCreate
74. * @ 功能说明: 为了方便管理，所有的任务创建函数都放在这个函数里面
75. * @ 参数 : 无
76. * @ 返回值 : 无
77. *****/
78. static void AppTaskCreate(void)
79. {
80.     BaseType_t xReturn = pdPASS; /* 定义一个创建信息返回值，默认为pdPASS */
81.
82.     taskENTER_CRITICAL(); /* 进入临界区 */
83.
84.     /* 创建LED_Task任务 */
85.     xReturn = xTaskCreate((TaskFunction_t)LED_Task, /* 任务入口函数 */
86.                          (const char*) "LED_Task", /* 任务名字 */
87.                          (uint16_t) 512, /* 任务栈大小 */
88.                          (void*) NULL, /* 任务入口函数参数 */
89.                          (UBaseType_t) 2, /* 任务的优先级 */
90.                          (TaskHandle_t*) &LED_Task_Handle); /* 任务控制块指针 */
91.
92.     if (pdPASS == xReturn)
93.         printf("创建LED_Task任务成功\n");
94.
95.     /* 创建Test_Task任务 */
96.     xReturn = xTaskCreate((TaskFunction_t)Test_Task, /* 任务入口函数 */
97.                          (const char*) "Test_Task", /* 任务名字 */
98.                          (uint16_t) 512, /* 任务栈大小 */
99.                          (void*) NULL, /* 任务入口函数参数 */
100.                         (UBaseType_t) 3, /* 任务的优先级 */
101.                         (TaskHandle_t*) &Test_Task_Handle); /* 任务控制块指针 */
102.
103.     if (pdPASS == xReturn)
104.         printf("创建Test_Task任务成功\n\n");
105.
106.     vTaskDelete(AppTaskCreate_Handle); /* 删除AppTaskCreate任务 */
107.
108.     taskEXIT_CRITICAL(); /* 退出临界区 */
109. }
110.
111. *****/
112. * @ 函数名 : LED_Task
113. * @ 功能说明: LED_Task任务主体
114. * @ 参数 : 无
115. * @ 返回值 : 无
116. *****/
117. static void LED_Task(void* parameter)
118. {
119.     while (1)
120.     {
121.         LED1_TOGGLE;
122.         vTaskDelay(1000); /* 延时1000个tick */
123.     }
124. }
125.
126. *****/
127. * @ 函数名 : Test_Task
128. * @ 功能说明: Test_Task任务主体
129. * @ 参数 : 无
130. * @ 返回值 : 无
131. *****/
132. static void Test_Task(void* parameter)
133. {
134.     uint32_t g_memsize;
135.     while (1)
136.     {
137.         if (Key_Scan(KEY1_GPIO_PORT, KEY1_GPIO_PIN) == KEY_ON )

```



```

138. {
139.     /* KEY1 被按下 */
140.     if(NULL == Test_Ptr)
141.     {
142.
143.         /* 获取当前内存大小 */
144.         g_memsize = xPortGetFreeHeapSize();
145.         printf("系统当前内存大小为 %d 字节, 开始申请内存\n", g_memsize);
146.         Test_Ptr = pvPortMalloc(1024);
147.         if(NULL != Test_Ptr)
148.         {
149.             printf("内存申请成功\n");
150.             printf("申请到的内存地址为%x\n", (int)Test_Ptr);
151.
152.             /* 获取当前内剩余存大小 */
153.             g_memsize = xPortGetFreeHeapSize();
154.             printf("系统当前内存剩余存大小为 %d 字节\n", g_memsize);
155.
156.             //向Test_Ptr中写入当数据:当前系统时间
157.             sprintf((char*)Test_Ptr, "当前系统TickCount = %d \n", xTaskGetTickCount());
158.             printf("写入的数据是 %s \n", (char*)Test_Ptr);
159.         }
160.     }
161.     else
162.     {
163.         printf("请先按下KEY2释放内存再申请\n");
164.     }
165. }
166. if( Key_Scan(KEY2_GPIO_PORT, KEY2_GPIO_PIN) == KEY_ON )
167. {
168.     /* KEY2 被按下 */
169.     if(NULL != Test_Ptr)
170.     {
171.         printf("释放内存\n");
172.         vPortFree(Test_Ptr);    //释放内存
173.         Test_Ptr=NULL;
174.         /* 获取当前内剩余存大小 */
175.         g_memsize = xPortGetFreeHeapSize();
176.         printf("系统当前内存大小为 %d 字节, 内存释放完成\n", g_memsize);
177.     }
178.     else
179.     {
180.         printf("请先按下KEY1申请内存再释放\n");
181.     }
182. }
183. vTaskDelay(20);/* 延时20个tick */
184. }
185. }
186.
187. /*****
188.  * @ 函数名   : BSP_Init
189.  * @ 功能说明: 板级外设初始化, 所有板子上的初始化均可放在这个函数里面
190.  * @ 参数     :
191.  * @ 返回值   : 无
192.  *****/
193. static void BSP_Init(void)
194. {
195.     /*
196.     * STM32中断优先级分组为4, 即4bit都用来表示抢占优先级, 范围为: 0~15
197.     * 优先级分组只需要分组一次即可, 以后如果有其他的任务需要用到中断,
198.     * 都统一用这个优先级分组, 千万不要再分组, 切忌。
199.     */
200.     NVIC_PriorityGroupConfig( NVIC_PriorityGroup_4 );
201.
202.     /* LED 初始化 */
203.     LED_GPIO_Config();
204.
205.     /* 串口初始化 */
206.     USART_Config();
207.
208.     /* 按键初始化 */
209.     Key_GPIO_Config();
210. }
211. }
212.
213. /*****END OF FILE*****/

```



五、内存管理的实验现象

程序编译好, 用 USB 线连接电脑和开发板的 USB 接口 (对应丝印为 USB 转串口), 用 DAP 仿真器把配套程序下载到 STM32 开发板, 在电脑上打开串口调试助手, 然后复位开发板, 我们按下 KEY1 申请内存, 然后按下 KEY2 释放内存, 可以在调试助手中看到串口打印信息与运行结果, 开发板的 LED 也在闪烁。

串口配置

端口COM13 USI

波特率115200

数据位8

停止位1

校验位无

操作● 关闭串口

接收区设置

☐ ASCII ☒ HEX

☐ 停止显示 ☐ 日志模式

这是一个[野火]-STM32全系列开发板-FreeRTOS内存管理实验
按下KEY1申请内存，按下KEY2释放内存
创建LED_Task任务成功
创建Test_Task任务成功

系统当前内存大小为 31960 字节，开始申请内存
内存申请成功
申请到的内存地址为0x20000388
系统当前内存剩余存大小为 30928 字节
写入的数据是 当前系统TickCount = 1591

释放内存
系统当前内存大小为 31960 字节，内存释放完成

CSDN @不秃也很强